DESIGN ANALYSIS AND ALGORITHM

WEEK 1 – ASSIGNMENT

1) Write a program to find the sum of first n natural numbers using user defined functions.

Program:

```
#include<stdio.h>
int sum=0,n;
int naturalnumbers(int n){
  for (int i=1;i<=n;i++){
    sum+=i;
  }
  return sum;
}
int main(){
  printf("enter the n value: ");
  scanf("%d",&n);
  int count=naturalnumbers(n);
  printf("The sum of %d natural numbers is: %d ",n,count);
}
```

Output:

```
enter the n value: 3
The sum of 3 natural numbers is: 6
```

Time complexity: O(n) The loop runs from 1 to n once.

Space complexity: O(1) Only fixed variables are used.

2) Write a program to find sum of square of first n natural numbers using user defined functions.

Program:

```
#include<stdio.h>
int sum=0,n;
int naturalnumbers(int n){
```

```
  for (int i=1;i<=n;i++){
    sum+=i*i;
  }
  return sum;
}
int main(){
  printf("enter the n value: ");
  scanf("%d",&n);
  int count=naturalnumbers(n);
  printf("The sum of %d natural numbers is: %d ",n,count);
}
```

Output:

```
enter the n value: 3
The sum of 3 natural numbers is: 14
```

Time complexity: O(n) Each number from 1 to n is squared and added.

Space complexity: O(1) No extra space apart from counters.

3) Write a program to find the sum of cubes of first n natural numbers.

Programs:

```
#include<stdio.h>
int sum=0,n;
int naturalnumbers(int n){
  for (int i=1;i<=n;i++){
    sum+=i*i*i;
  }
  return sum;
}
int main(){
  printf("enter the n value: ");
  scanf("%d",&n);
  int count=naturalnumbers(n);
  printf("The sum of %d natural numbers is: %d ",n,count);
}
```

Output:

```
enter the n value: 3
The sum of 3 natural numbers is: 36
```

Time complexity: O(n) Loop iterates n times to compute cubes.

Space complexity: O(1) Uses constant extra memory.

4) Write a program to find factorial of n natural number using recursive function.

Program:

```
#include <stdio.h>
int fib(int n) {
  if (n == 0)
    return 0;
  else if (n == 1)
    return 1;
  else
    return fib(n - 1) + fib(n - 2);
}

int main() {
  int n;
  printf("enter n value: ");
  scanf("%d",&n);
  printf("Fibonacci of %d is: %d\n", n, fib(n));
  return 0;
}
```

Output:

```
enter n value: 3
Fibonacci of 3 is: 2
```

Time complexity: O(2^n) Each call generates two more recursive calls.

Space complexity: O(n) Recursion depth grows up to n.

5) Write a program to transpose a 3X3 matrix.

Program:

```
#include <stdio.h>
int main() {
   int a[3][3], transpose[3][3];
   int i, j;
   printf("Enter elements: \n");
   for (i = 0; i < 3; i++) {
      for (j = 0; j < 3; j++) {
         scanf("%d", &a[i][j]);
      }
   }
   for (i = 0; i < 3; i++) {
      for (j = 0; j < 3; j++) {
         transpose[j][i] = a[i][j];
      }
   }
   printf("Transpose of 3x3 matrix is:\n");
   for (i = 0; i < 3; i++) {
      for (j = 0; j < 3; j++) {
         printf("%d ", transpose[i][j]);
      }
      printf("\n");
   }
   return 0;
}
```

Output:

```
Enter elements:
1 2 3 4 5 6 7 8 9
Transpose of 3x3 matrix is:
1 4 7
2 5 8
3 6 9
```

Time complexity: O(1) Only 9 fixed operations for a 3×3 matrix.

Space complexity: O(1) Uses a fixed 3×3 matrix for output.

6) Write a program to print the Fibonacci series upto a given number using user defined function (for first n natural numbers).

Program:

```
#include <stdio.h>
void printFibonacci(int n) {
   int a = 0, b = 1, c;
   printf("Fibonacci Series: ");
   for (int i = 1; i <= n; i++) {
      printf("%d ", a);
      c = a + b;
      a = b;
      b = c;
   }
}
int main() {
   int n;

   printf("Enter how many Fibonacci numbers you want: ");
   scanf("%d", &n);
   printFibonacci(n);
   return 0;
}
```

Output:

```
enter n value: 3
Fibonacci of 3 is: 2
```

Time complexity: O(n) Single loop runs n times.

Space complexity: O(1) Only three variables store results.